

PROJECT MANIFEST

Wetground CMS — JSON First, Component Driven

1. Vision

Ce projet est un **CMS léger, orienté composants, basé sur des fichiers JSON**.

Le système sépare strictement : - **Structure** (pages) - **Contenu** (articles, galeries, etc.) - **Rendu** (PHP / HTML)

Aucun contenu ne contient de HTML. Le HTML est exclusivement généré par les vues.

Objectifs principaux : - Portabilité maximale - Versionnage Git-friendly - Édition simple via une interface admin - Architecture compréhensible sans framework

2. Règles Fondamentales

Données

- Toutes les données de contenu sont stockées en JSON
- Aucun HTML dans les fichiers JSON
- Tous les champs textuels sont multilingues
- Les clés de langue sont explicites (`fr`, `en`, `nl`)

Architecture

- Une **Page** ne contient jamais directement du contenu textuel
- Une **Page** référence des **Composants**
- Un **Article** est un composant
- Un composant est composé de **Blocs**

Admin

- L'admin ne manipule jamais du HTML
 - L'admin assemble des structures JSON valides
 - Toute sortie admin doit respecter les schémas définis
-

3. Hiérarchie Conceptuelle

```
Site
├── Pages
│   ├── Components
│   │   ├── Articles
│   │   └── Blocks
```

4. Structure des Dossiers

```
/json
  /pages
  /articles
  /galleries

/admin
  /pages
  /articles
  /components

/src
  /model
  /view
  /controller

/inc
  head.php
  header.php
  main.php
  footer.php
```

5. Schémas JSON Officiels

5.1 Page

Une page est un conteneur de composants

```
{
  "type": "page",
  "meta": {
    "id": "openatelier",
    "title": {
```

```

    "fr": "Open Atelier",
    "en": "Open Studio",
    "nl": "Open Atelier"
  }
},
"content": [
  { "type": "article", "ref": "intro-openatelier" },
  { "type": "gallery", "ref": "atelier-2025" }
]
}

```

5.2 Article

Un article est un composant structuré en blocs

```

{
  "type": "article",
  "meta": {
    "id": "intro-openatelier",
    "author": "admin",
    "created": "2026-01-06",
    "status": "published"
  },
  "content": [
    {
      "type": "title",
      "level": 2,
      "text": {
        "fr": "Bienvenue",
        "en": "Welcome",
        "nl": "Welkom"
      }
    },
    {
      "type": "text",
      "content": {
        "fr": "Texte français",
        "en": "English text",
        "nl": "Nederlandse tekst"
      }
    }
  ]
}

```

6. Blocs Supportés

title

```
{
  "type": "title",
  "level": 2,
  "text": { "fr": "", "en": "", "nl": "" }
}
```

text

```
{
  "type": "text",
  "content": { "fr": "", "en": "", "nl": "" }
}
```

list

```
{
  "type": "list",
  "style": "ul",
  "items": [
    { "fr": "", "en": "", "nl": "" }
  ]
}
```

link

```
{
  "type": "link",
  "label": { "fr": "", "en": "", "nl": "" },
  "url": "",
  "target": "_blank"
}
```

gallery

```
{
  "type": "gallery",
  "ref": "atelier-2025"
}
```

7. Rendu Frontend

Principes

- Chaque type de bloc a son renderer PHP dédié
- Le renderer décide des balises HTML
- Le JSON ne contient aucune logique d'affichage

Exemple

```
function renderBlock($block, $lang) {
    switch ($block['type']) {
        case 'title':
            echo '<h' . $block['level'] . '>' . htmlspecialchars($block['text']
[$lang]) . '</h' . $block['level'] . '>';
            break;
        case 'text':
            echo '<p>' . nl2br(htmlspecialchars($block['content'][$lang])) .
'</p>';
            break;
    }
}
```

8. Philosophie Admin

L'admin est un **générateur de structures**, pas un éditeur de HTML.

L'utilisateur admin : - Crée des articles - Ajoute des blocs - Réorganise les blocs - Attache des composants à des pages

L'admin garantit que chaque fichier JSON est valide et conforme au schéma.

9. Navigation

Le fichier `json/menus.json` est la référence pour : - Pages existantes - Ordre d'affichage - Titres multilingues

L'admin s'appuie sur ce fichier pour : - Proposer les pages à éditer - Empêcher l'édition de pages non déclarées

10. Conventions Techniques

PHP

- Pas de framework
- Pas d'autoload
- Includes explicites
- Chemins basés sur

Sécurité

- Whitelist des pages admin
 - Session obligatoire
 - Protection basename() sur tous les fichiers
-

11. Non-Objectifs

- Pas de WYSIWYG
 - Pas d'éditeur HTML
 - Pas de base SQL
 - Pas de dépendances JS lourdes
-

12. État Actuel

- Architecture définie
 - Pages JSON opérationnelles
 - Menu JSON utilisé comme référence
 - Admin pages fonctionnel
 - Composant Article en cours de formalisation
-

13. TODO LIST

Priorité Haute

- [] Créer admin des Articles
- [] CRUD des blocs (ajout / suppression / réorganisation)
- [] Sélecteur de langue live dans l'admin
- [] Validation JSON automatique

Priorité Moyenne

- [] Admin des Galeries
- [] Système d'attachement composants → pages

- [] Drag & drop pour l'ordre des blocs

Priorité Basse

- [] Versioning interne des articles
 - [] Prévisualisation frontend
 - [] Historique de modifications
-

14. Décisions Clés

- JSON préféré à SQL pour portabilité et versionnage
 - Séparation stricte contenu / rendu
 - Page = composition de composants
 - Article = unité éditoriale principale
-

15. Règle d'Or

Si une information peut être déplacée hors du HTML, elle doit l'être.

16. Continuité IA

Toute IA reprenant ce projet doit : - Respecter les schémas définis - Ne pas introduire de HTML dans les données - Maintenir la hiérarchie Page → Component → Block - Ne pas introduire de base de données

Fin du manifeste